

StudyFlow Offline AI — Architecture Based on Your Actual Project

1. Your Current StudyFlow Architecture

From the ZIP, your project currently looks roughly like this:

```
student/
├── main.py
├── components/
│   ├── header.py
│   ├── sidebar.py
│   ├── widgets.py
│   └── quick_search.py
├── modules/
│   ├── dashboard/
│   ├── subjects/
│   ├── assignments/
│   ├── attendance/
│   ├── calendar/
│   ├── cgpa/
│   ├── documents/
│   ├── notes/
│   ├── planner/
│   ├── productivity/
│   ├── settings/
│   └── timetable/
├── database/
│   ├── database.py
│   ├── db_setup.py
│   ├── queries.py
│   └── backup_manager.py
├── utils/
│   ├── file_manager.py
│   ├── logger.py
│   └── theme.py
├── UserData/
│   ├── studyflow.db
│   └── Files/
└── requirements / configuration
```

Your actual file architecture is already suitable for our AI.

2. What We Are Adding

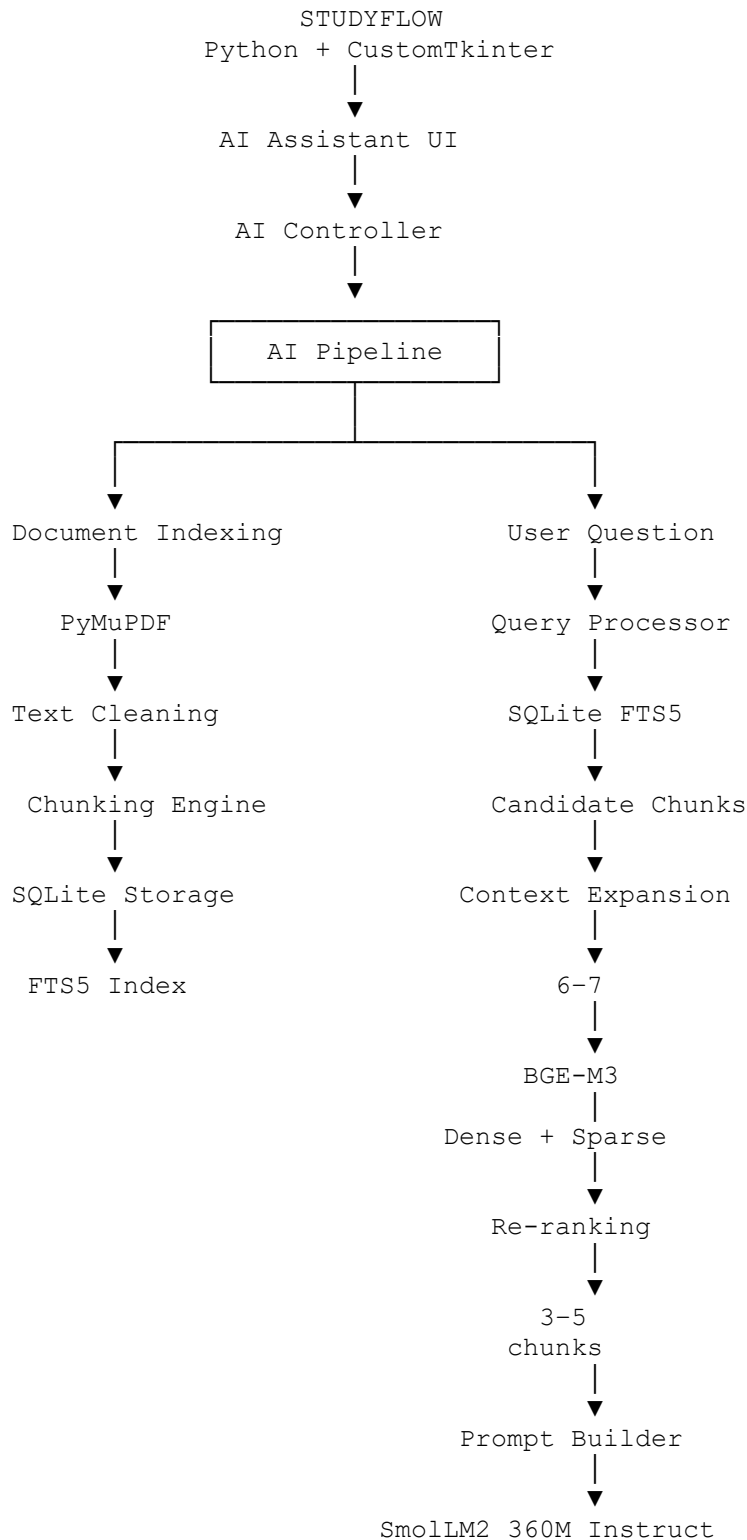
I recommend adding a completely separate AI subsystem:

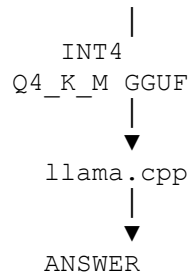
```
student/
├── ai/
│   ├── pdf/
│   │   ├── extractor.py
│   │   ├── cleaner.py
│   │   └── chunker.py
│   ├── retrieval/
│   │   ├── fts_search.py
│   │   ├── embedding.py
│   │   ├── hybrid_search.py
│   │   └── ranker.py
│   ├── generation/
│   │   ├── prompt.py
│   │   └── generator.py
│   ├── models/
│   │   ├── bge-m3/
│   │   └── smollm2/
│   ├── pipeline.py
│   └── config.py
└── modules/
    └── ai_assistant/
        └── assistant.py
```

This keeps the AI **isolated from your existing modules**.

3. The Complete Architecture

The final system becomes:





4. First Important Decision: PDF Storage

Your existing system already does this correctly.

I inspected your `utils/file_manager.py`.

Your PDFs are stored like:

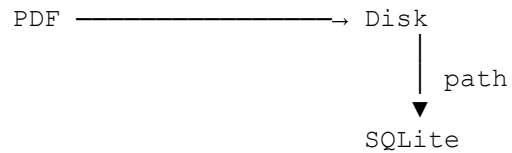
```
UserData/
├── Files/
│   ├── 1/
│   │   ├── Dcn practicals.pdf
│   │   └── IS2 ALA1.pdf
│   ├── 4/
│   │   └── Dcn practicals.pdf
│   └── 7/
│       └── IS2 ALA1.pdf
```

And SQLite stores metadata through your existing `subject_files` table:

```
subject_files
-----
id
subject_id
file_name
file_path
file_type
file_size
uploaded_at
```

We KEEP this.

We don't put PDF binary data into SQLite.



That's exactly what we want for the low-RAM design.

5. What Happens When User Uploads a PDF?

Currently:

```
User
↓
Documents module
↓
file_manager.save_file()
↓
UserData/Files/{subject_id}/
↓
subject_files table
```

We add one step:

```
User
↓
Documents module
↓
Save PDF to disk
↓
SQLite metadata
↓
AI indexing queue
↓
PDF extraction
```

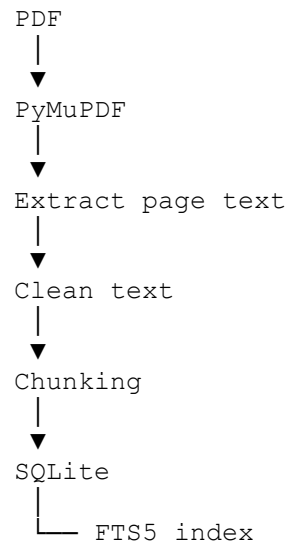
So the existing Documents module doesn't need to become an AI module.

It simply tells the AI subsystem:

"A new PDF was added."

6. AI Indexing Pipeline

This happens when a PDF is added.



This is the **indexing phase**.

It doesn't happen every time the user asks a question.

7. PyMuPDF

We'll use:

PyMuPDF

for PDF extraction.

It reads the PDF page-by-page.

For example:

Page 1

↓

Text

Page 2

↓

Text

Page 3

↓

Text

We keep the page number.

That's important because later StudyFlow can tell the user:

Source: Data Structures.pdf — Page 42

8. Chunking

The extracted text goes into our chunking engine.

Our initial configuration:

Target: ~400 tokens
Overlap: ~60 tokens
Strategy: paragraph/heading aware

So:

PDF
↓
Page text
↓
Paragraphs
↓
~400-token chunks

Example:

Chunk 001
Page: 1
Position: 0

Chunk 002
Page: 1-2
Position: 1

Chunk 003
Page: 2
Position: 2

Each chunk should have an ID.

9. We Need New SQLite AI Tables

This is one of the biggest changes to your current database.

I recommend **not modifying your existing** `subject_files` **table to store AI chunks.**

Instead, add dedicated AI tables.

ai_documents

```
ai_documents
-----
id
file_id
status
total_pages
total_chunks
indexed_at
error_message
```

Relationship:

```
subject_files
  |
  | file_id
  ▼
ai_documents
```

This lets us know whether a PDF has been indexed.

10. ai_chunks

ai_chunks

id
ai_document_id
chunk_index
page_start
page_end
text
token_count

Example:

```
id: 1842
document_id: 17
chunk_index: 421
page_start: 42
page_end: 43
token_count: 397
text: "Quick sort is..."
```

This is the actual searchable text.

11. FTS5

Now comes our first retrieval layer.

We'll create an SQLite FTS5 virtual table conceptually like:

```
ai_chunks_fts
-----
chunk_id
text
```

It indexes the chunk text.

So when the user asks:

"What is the time complexity of quick sort?"

we don't scan all 5,000 pages manually.

Instead:

```
Question
  ↓
FTS5
  ↓
Relevant chunk IDs
```

This is extremely important for our low-resource design.

12. Question Processing

Now suppose the user asks:

"Explain the partition process in quick sort."

The AI pipeline receives:

```
query =  
"Explain the partition process in quick sort."
```

The first thing we do is **not** send it to SmolLM2.

We search.

```
Question  
  |  
  ▼  
FTS5  
  |  
  ▼  
Candidate chunks
```

13. Candidate Retrieval

Suppose FTS5 returns:

```
Chunk 420  
Chunk 421  
Chunk 733  
Chunk 1201
```

Now we don't immediately send them to the SLM.

We perform **context expansion**.

14. Context Expansion — Your Original Idea

This is one of the best parts of the architecture you proposed earlier.

Suppose:

Chunk 421

matches.

Instead of using only:

421

we retrieve nearby chunks:

418
419
420
421 ← matched
422
423
424

Now the semantic retrieval stage sees approximately:

6–7 chunks.

This solves the problem where the relevant answer begins in one chunk and finishes in another.

15. BGE-M3

Now BGE-M3 enters.

Our chosen primary embedding model is:

BGE-M3

This is particularly valuable because multilingual retrieval is one of our goals.

It supports 100+ languages and provides 1024-dimensional embeddings.

So:

```
Question
  ↓
BGE-M3
  ↓
Question embedding
```

and:

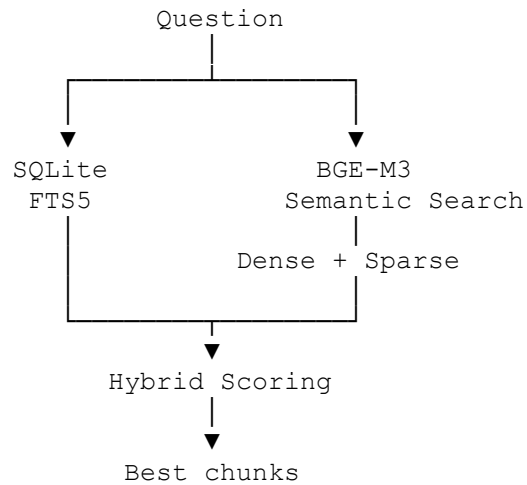
```
Candidate chunk
  ↓
BGE-M3
  ↓
Chunk embedding
```

Then we compare them.

16. Dense + Sparse Retrieval

BGE-M3 gives us an opportunity to use more than traditional vector similarity.

Our retrieval architecture becomes:



This is much better than simply saying:

"Put everything into a vector database."

We aren't doing that.

17. No Vector Database

This remains one of our key architectural decisions.

We are **not** building StudyFlow around:

```
PDF
↓
500,000 vectors
↓
Vector DB
↓
Huge RAM consumption
```

Instead:

```
PDF
↓
SQLite chunks + FTS5
↓
small candidate set
↓
BGE-M3
↓
temporary embeddings
↓
ranking
↓
discard temporary vectors
```

That fits our low-RAM philosophy much better.

18. Important BGE-M3 Optimization

There is one thing I want to make explicit after inspecting your project:

We should NOT immediately run BGE-M3 over every chunk during every question.

Instead:

```
5,000-page PDF
      ↓
FTS5
      ↓
small candidate set
      ↓
context expansion
      ↓
6-7 chunks
      ↓
BGE-M3
```

This is the architecture I want us to implement.

19. BGE-M3 Runtime

We previously decided:

Development/testing

```
BGE-M3
      ↓
PyTorch
```

Production target

```
BGE-M3
      ↓
ONNX
      ↓
ONNX Runtime
```

Why?

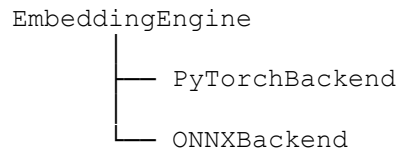
Because StudyFlow is supposed to be lightweight.

We don't want to ship a huge PyTorch runtime if we can achieve the same inference with a smaller production runtime.

But:

We should benchmark ONNX against PyTorch before making the production conversion final.

The architecture should therefore have an abstraction:



That is much better engineering than hard-coding BGE-M3 directly into the application.

20. Ranking

After BGE-M3 evaluates the candidates, we might have something like:

```
Chunk 421 → 0.94
Chunk 420 → 0.91
Chunk 422 → 0.88
Chunk 419 → 0.84
Chunk 423 → 0.73
Chunk 424 → 0.41
```

Then select the best context.

Our target:

```
6-7 candidate chunks
      ↓
3-5 final chunks
```

21. Now SmolLM2 Finally Enters

This is where our **small language model** comes in.

Chosen model:

SmolLM2 360M Instruct

Its job is **not searching**.

Its job is:

```
Question
+
Relevant context
↓
Understand
↓
Generate answer
```

That's a very important separation.

22. The SLM Does NOT Read the PDF

This:

PDF → SmolLM2

is **NOT** our architecture.

Instead:

```
PDF
↓
Index
↓
Search
↓
Retrieve
↓
BGE-M3
↓
3-5 relevant chunks
↓
SmolLM2
```

The SLM only sees the information it actually needs.

23. SmolLM2 Quantization

Our decision remains:

INT4

And for the local GGUF deployment:

Q4_K_M

So:

```
SmolLM2 360M Instruct
  ↓
  INT4
  ↓
  Q4_K_M
  ↓
  GGUF
  ↓
  llama.cpp
```

This is our low-memory generation path.

24. Why llama.cpp?

Because PyTorch isn't the best choice for the final SLM runtime when our priority is:

- CPU inference
- low RAM
- local execution
- quantized GGUF models
- simple deployment

So:

BGE-M3
→ ONNX Runtime

SmolLM2
→ GGUF
→ llama.cpp

That's a clean separation.

25. Prompt Construction

Before SmolLM2 runs, we build a controlled prompt.

Conceptually:

```
SYSTEM:
You are StudyFlow's offline academic assistant.
Answer using only the provided context.
If the answer is not present, say that you cannot find it
in the uploaded document.
```

```
CONTEXT:
```

```
[Page 42]
...chunk...
```

```
[Page 43]
...chunk...
```

```
[Page 44]
...chunk...
```

```
QUESTION:
```

```
Explain the partition process in quick sort.
```

Then:

```
Prompt
```

```
↓
```

```
SmolLM2 360M Q4_K_M
```

26. Answer

The model generates:

```
Quick sort uses partitioning to divide the array...
```

StudyFlow can additionally display:

```
Sources:
```

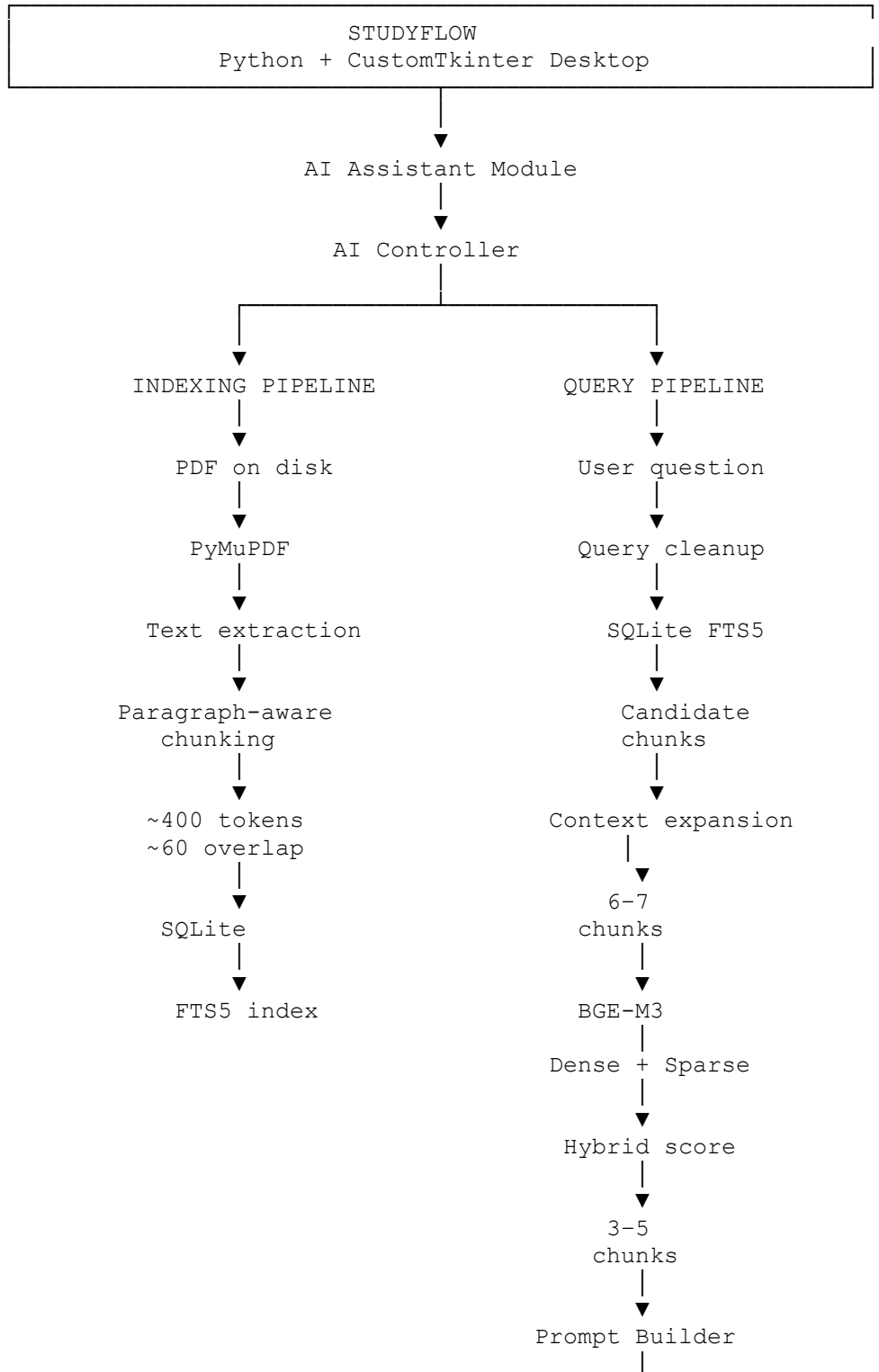
```
 Data Structures.pdf
```

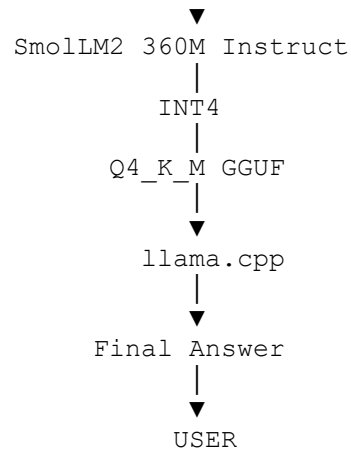
```
 Pages 42-44
```

This makes the feature much more useful academically.

27. Complete Runtime Flow

This is the **actual final architecture I recommend for your project**:





28. Where Everything Lives

I would organize your actual project like this:

```
student/
├── ai/
│   ├── __init__.py
│   ├── pipeline.py
│   ├── config.py
│   ├── pdf/
│   │   ├── __init__.py
│   │   ├── extractor.py
│   │   ├── cleaner.py
│   │   └── chunker.py
│   ├── retrieval/
│   │   ├── __init__.py
│   │   ├── fts_search.py
│   │   ├── embedding.py
│   │   ├── hybrid_search.py
│   │   └── ranker.py
│   ├── generation/
│   │   ├── __init__.py
│   │   ├── prompt.py
│   │   └── generator.py
│   └── models/
│       ├── bge-m3/
│       └── smollm2/
└── modules/
    └── ai_assistant/
        ├── __init__.py
```

```
└─ assistant.py
├─ database/
│   ├── database.py
│   ├── db_setup.py
│   └── queries.py
└─ UserData/
    ├── studyflow.db
    ├── Files/
    └── AI/
```

29. What Goes in `UserData/AI`

I'd keep generated AI data separate:

```
UserData/
├─ studyflow.db
├─ Files/
│   ├── 1/
│   │   └─ Dcn practicals.pdf
│   ├── 7/
│   │   └─ IS2 ALA1.pdf
└─ AI/
    ├── cache/
    ├── index/
    └── models/
```

However, **the model files themselves may be better kept outside** `UserData` depending on packaging.

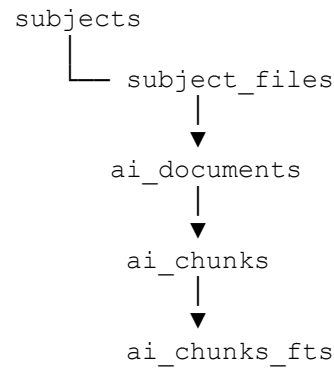
For example:

```
student/
└─ ai/
    └─ models/
        ├── bge-m3/
        └── smollm2-360m-q4_k_m.gguf
```

while user-generated indexes/cache stay under `UserData/AI`.

30. Database Relationship

Your existing database:



So the AI doesn't duplicate the actual PDF.

It only creates an **AI-readable representation** of the PDF.

31. New Tables

I'd add:

ai_documents

```
CREATE TABLE ai_documents (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  file_id INTEGER NOT NULL UNIQUE,  
  status TEXT DEFAULT 'pending',  
  total_pages INTEGER DEFAULT 0,  
  total_chunks INTEGER DEFAULT 0,  
  indexed_at TIMESTAMP,  
  error_message TEXT,  
  FOREIGN KEY (file_id)  
    REFERENCES subject_files(id)  
    ON DELETE CASCADE  
);
```

ai_chunks

```
CREATE TABLE ai_chunks (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    ai_document_id INTEGER NOT NULL,  
    chunk_index INTEGER NOT NULL,  
    page_start INTEGER,  
    page_end INTEGER,  
    text TEXT NOT NULL,  
    token_count INTEGER,  
    FOREIGN KEY (ai_document_id)  
        REFERENCES ai_documents(id)  
        ON DELETE CASCADE  
);
```

Then FTS5 indexes `ai_chunks.text`.

32. What We DON'T Store

This is important for keeping the system lightweight.

We don't need to permanently store:

- ✗ PDF binary inside SQLite
- ✗ Entire PDF text in RAM
- ✗ Every BGE-M3 embedding in RAM
- ✗ Huge vector database
- ✗ Cloud embeddings
- ✗ Cloud LLM responses

Instead:

PDF → Disk

Metadata → SQLite

Chunks → SQLite

Search index → SQLite FTS5

BGE vectors → temporary during retrieval

SLM → local quantized model

33. Memory Architecture

This is the part that matches the original goal you had.

Imagine a 5,000-page PDF.

We **don't** do:

5,000 pages
↓
RAM
↓
BGE-M3
↓
Huge vector database

We do:

5,000 pages
↓
Disk
↓
SQLite chunks
↓
FTS5
↓
~few candidate chunks
↓
6-7 chunks
↓
BGE-M3
↓
3-5 chunks
↓
SmolLM2

Therefore the **document size is mostly a storage/indexing concern rather than an inference-context concern.**

34. Time Complexity

Our earlier complexity discussion still applies, but now we can map it directly to the real architecture.

PDF indexing

Extraction	$O(T)$
Chunking	$O(T)$
FTS indexing	approximately linear in indexed text

where T = total extracted text.

Question answering

FTS5	→ indexed lookup
Context expansion	→ $O(1)$
BGE-M3	→ $O(C \times L)$
Similarity	→ $O(C \times 1024)$
Ranking	→ $O(C \log C)$
SLM generation	→ dominant runtime

where:

$C \approx 6-7$ chunks
 $L \approx 400$ tokens

So the expensive part is primarily **local model inference**, not searching a 5,000-page PDF.

35. The "5 Seconds" Goal

For the actual StudyFlow implementation, I'd make this a **performance target**, not a guaranteed claim yet:

Target typical response time: approximately 3-5 seconds on a suitable 8 GB RAM system.

We'll only advertise something like:

"Average response time: ~5 seconds"

after we benchmark the completed implementation.

We'll measure:

FTS5 time
BGE-M3 time
Ranking time
Prompt construction time
SmolLM2 generation time
Total time
RAM peak

That will give us real numbers.

36. Final Technology Stack — Corrected for Your Actual Project

Layer	Technology
Application	Python
GUI	CustomTkinter
Database	SQLite
Existing file storage	Local filesystem
PDF extraction	PyMuPDF
Text cleaning	Python
Chunking	Custom Python
Chunk size	~400 tokens
Chunk overlap	~60 tokens
Keyword retrieval	SQLite FTS5
Semantic retrieval	BGE-M3
BGE development	PyTorch
BGE production target	ONNX Runtime
Retrieval	Hybrid dense + sparse
Candidate context	6 - 7 chunks
Final context	3 - 5 chunks
Generator	SmolLM2 360M Instruct
Quantization	INT4
Model format	GGUF Q4_K_M
SLM runtime	llama.cpp
Vector DB	None
Cloud API	None

Layer	Technology
Internet	Not required

37. The One-Sentence Architecture

If you need to explain StudyFlow AI to someone quickly:

StudyFlow AI is a completely offline RAG-based PDF assistant that stores PDFs on disk, indexes extracted text in SQLite/FTS5, uses BGE-M3 for lightweight multilingual hybrid retrieval of only a few relevant chunks, and uses a locally quantized SmolLM2 360M INT4 model through llama.cpp to generate the final answer.

And the key design principle is:

The PDF can be huge; the AI context stays small.